Use Case: Evaluating an AWS Code-Generation Prompt

The five companion notebooks (`001` through `005`) are a single running example that shows how to take the five-step eval workflow from 02_A_typical_eval_workflow.md and actually build it, one capability at a time.

The Scenario

Imagine you're building an internal tool that lets engineers describe an AWS-related task in plain English and get back working code — a Python function, a JSON config, or a regex — generated by Claude.

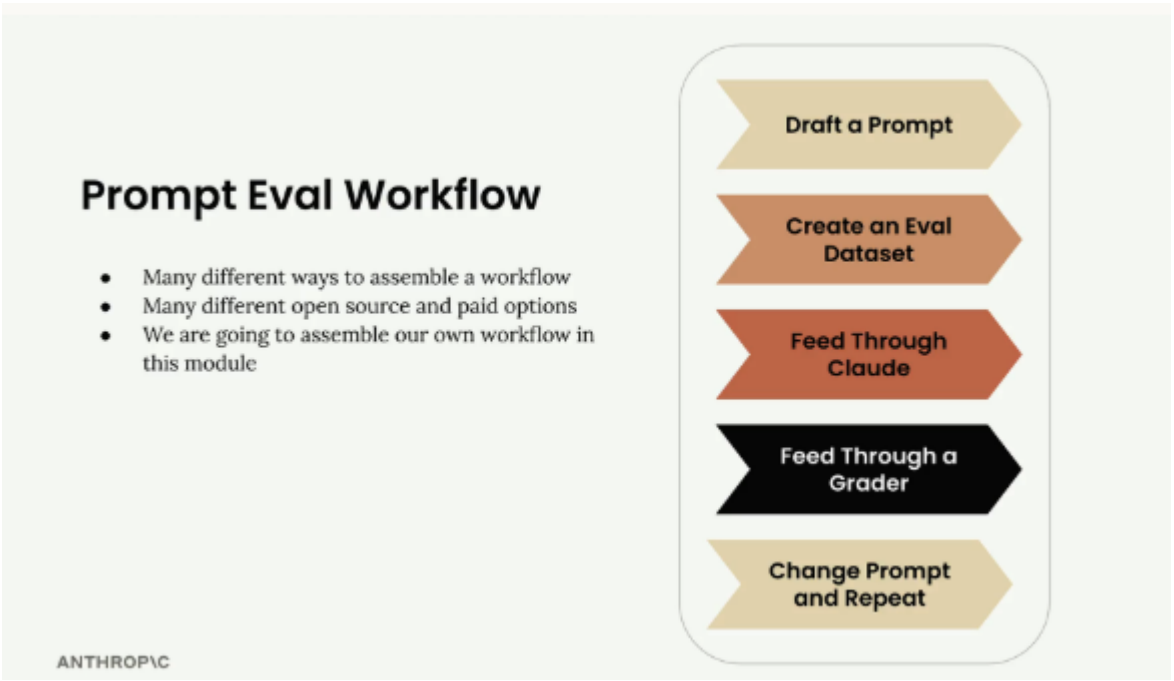
Before shipping this to your team, you need a real answer to: **"How good is this prompt, actually?"** Not a vibe from testing it three times yourself, but a repeatable, scorable measurement you can rerun every time you tweak the prompt.

That's the use case these notebooks walk through, end to end.

A Typical Eval Workflow

A typical prompt evaluation workflow follows five key steps that help you systematically improve your prompts through objective measurement.

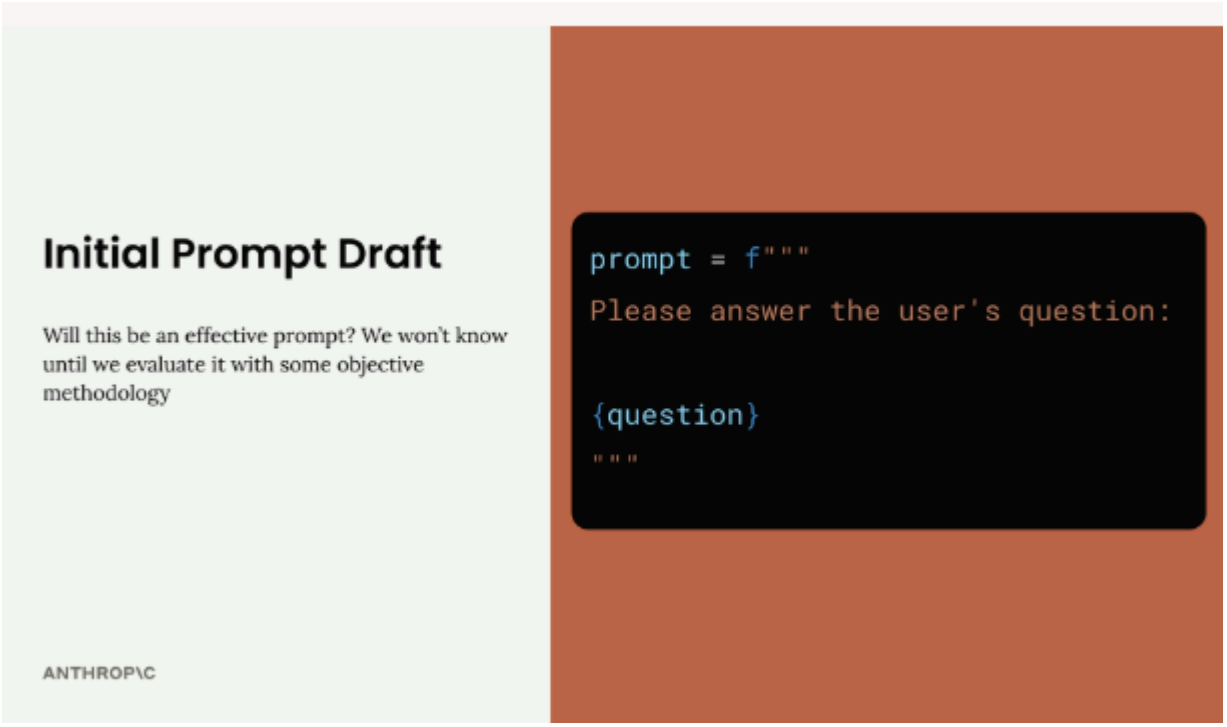
While there are many different ways to assemble these workflows and various open source and paid tools available, understanding the core process helps you start small and scale up as needed.



Step 1: Draft a Prompt

Start by writing an initial prompt that you want to improve.For this example, we'll use a simple prompt:

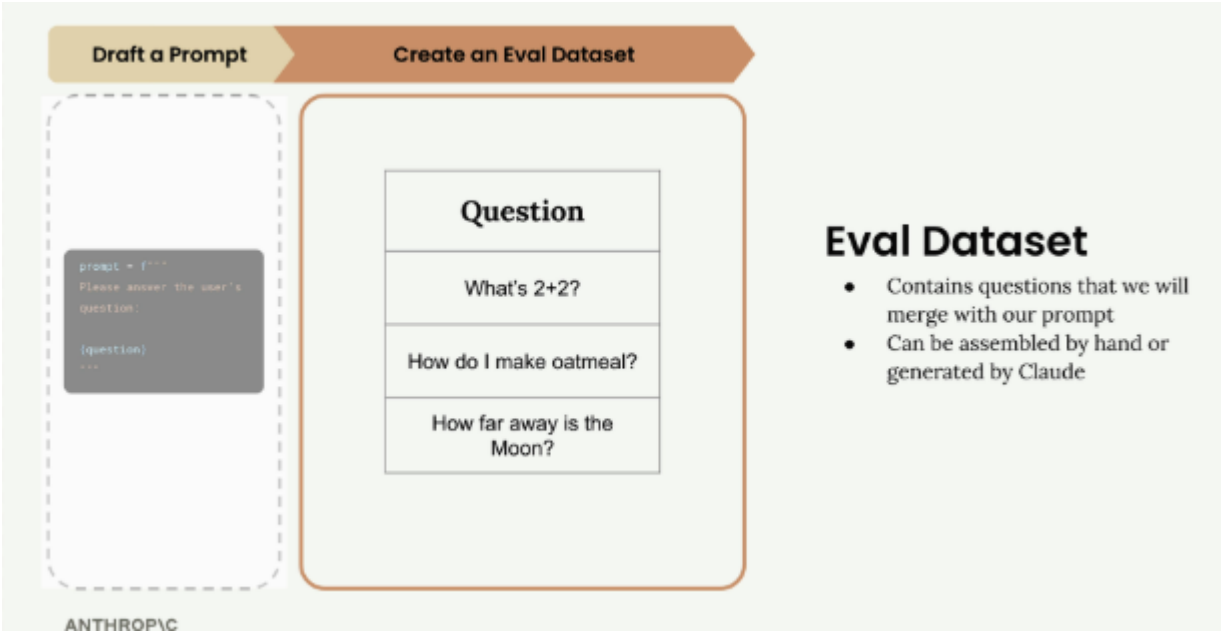
```
prompt = f"""
Please answer the user's question:

{question}
"""
```



This basic prompt will serve as our baseline for testing and improvement.

Step 2: Create an Eval Dataset

Your evaluation dataset contains sample inputs that represent the types of questions or requests your prompt will handle in production. The dataset should include questions that will be interpolated into your prompt template.



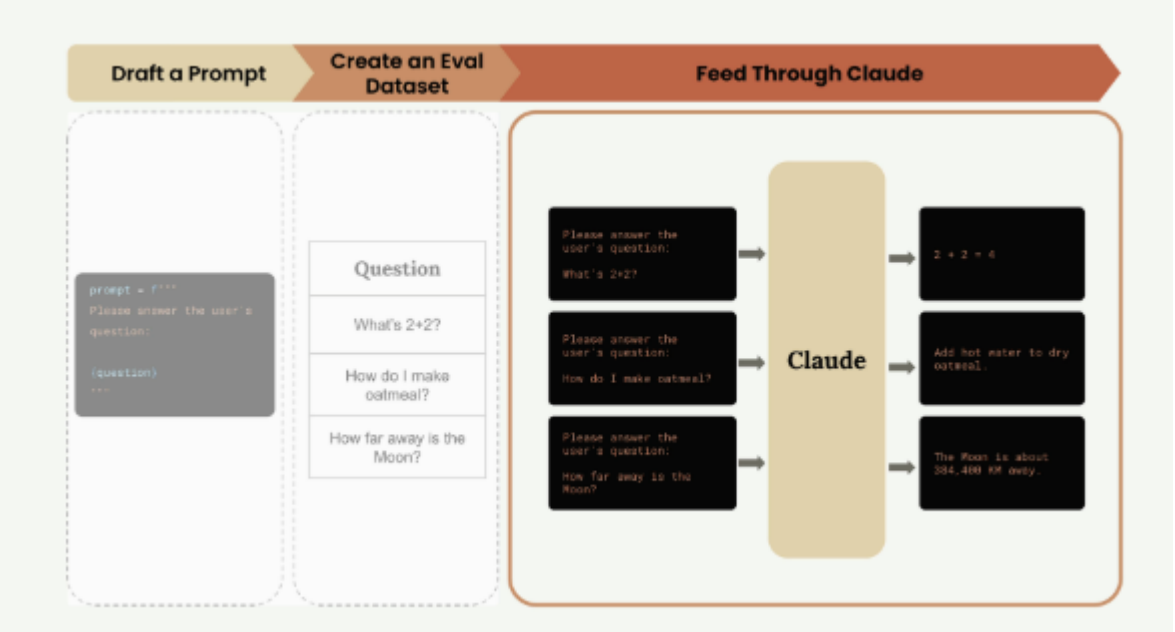
For this example, our dataset includes three questions:

- "What's 2+2?",
- "How do I make oatmeal?",
- "How far away is the Moon?"

In real-world evaluations, you might have tens, hundreds, or even thousands of records.You can assemble these datasets by hand or use Claude to generate them for you.

Step 3: Feed Through Claude

Take each question from your dataset and merge it with your prompt template to create complete prompts.Then send each one to Claude to get responses.



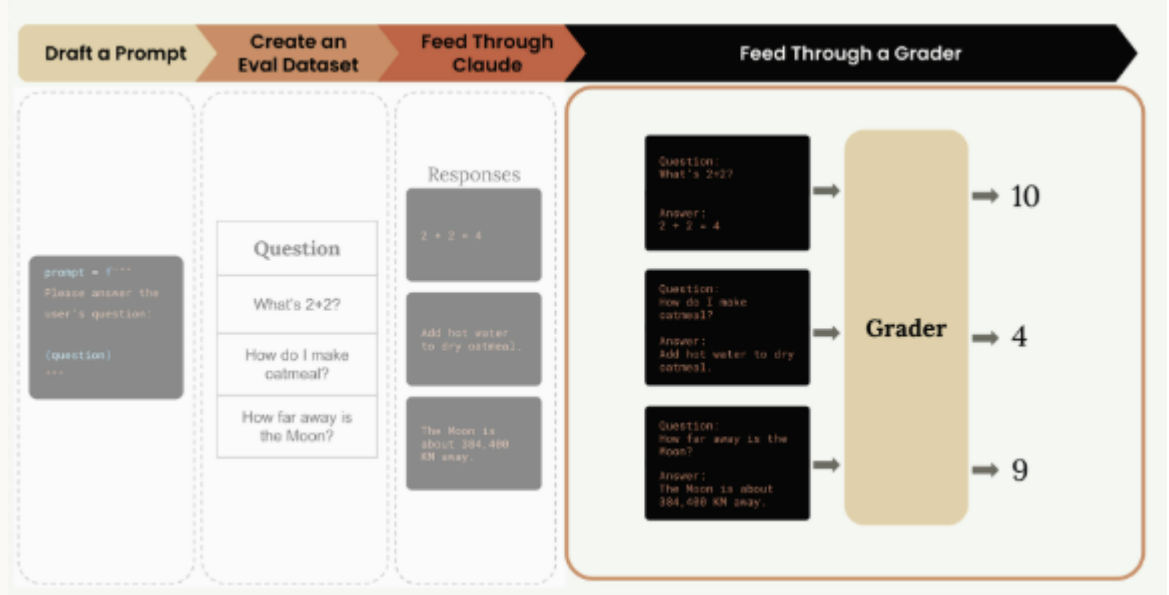
For example, the first question becomes:

```
Please answer the user's question:

What's 2+2?
```

Claude might respond with: "2 + 2 = 4" for the math question, provide oatmeal cooking instructions for the second question, and give the distance to the Moon for the third.

Step 4: Feed Through a Grader

The grader evaluates the quality of Claude's responses by examining both the original question and Claude's answer.This step provides objective scoring, typically on a scale from 1 to 10, where 10 represents a perfect answer and lower scores indicate room for improvement.



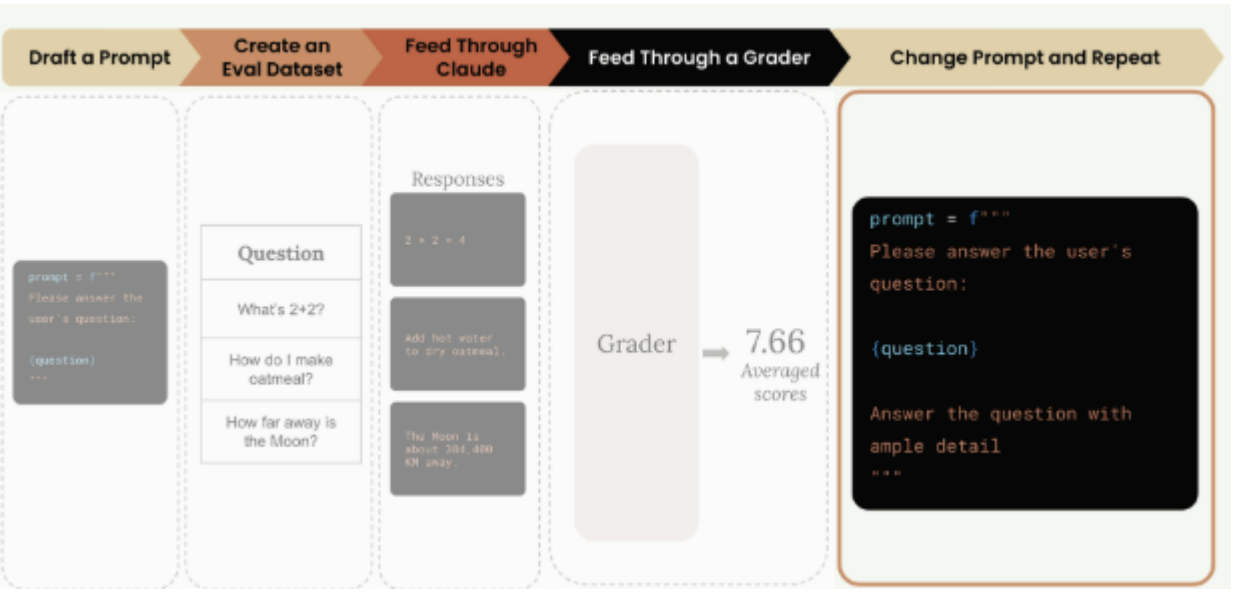
In our example, the grader might assign:

- Math question: 10 (perfect answer)
- Oatmeal question: 4 (needs improvement)
- Moon question: 9 (very good answer)

This average score across all questions gives you an objective measurement: (10 + 4 + 9) / 3 = 7.67

Step 5: Change Prompt and Repeat

Now that you have a baseline score, you can modify your prompt and run the entire process again to see if your changes improve performance.



Example:

```
prompt = f"""
Please answer the user's question:
```

```
{question}

Answer the question with ample detail
"""
```

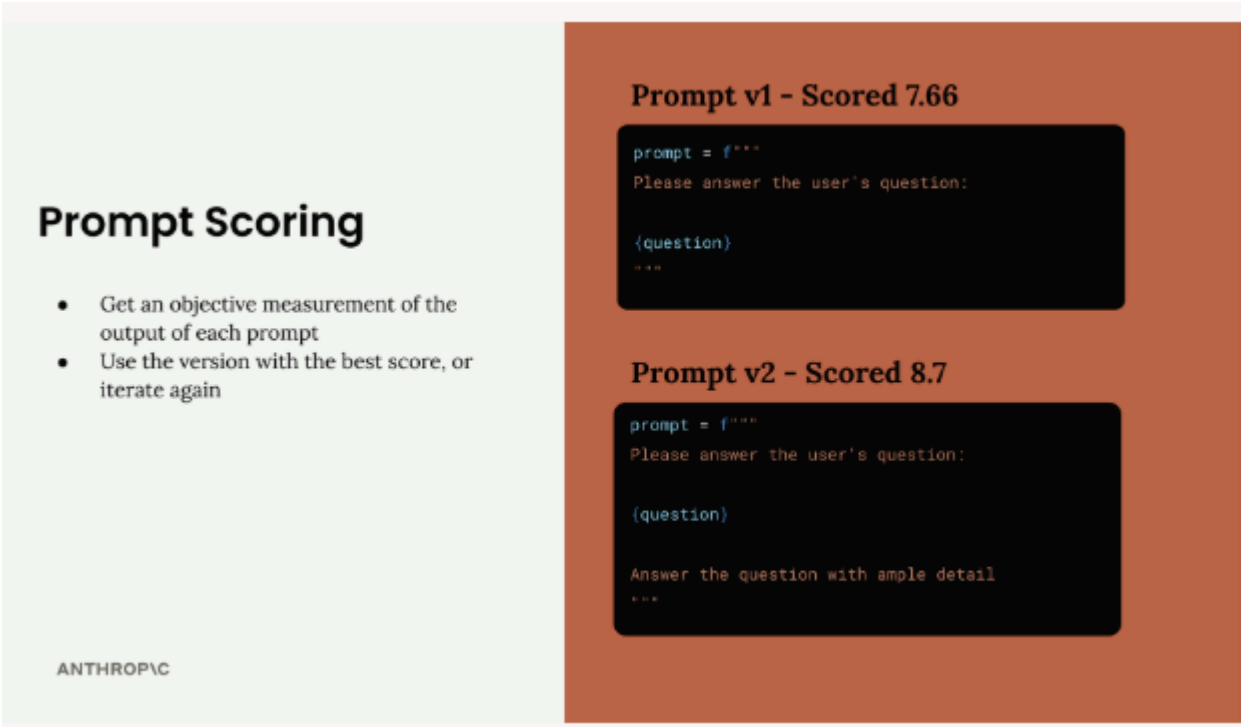
After running this improved prompt through the same evaluation process, you might get a higher average score of 8.7, indicating that the additional instruction helped Claude provide better responses.

Prompt Scoring

The key benefit of this workflow is getting objective measurements of prompt performance.

You can:

- Compare different prompt versions numerically
- Use the version with the best score
- Continue iterating to find even better approaches



This systematic approach removes guesswork from prompt engineering and gives you confidence that your changes are actually improvements rather than just different variations.

Notebook 1 — `001_prompt_evals.ipynb`: Building the Eval Dataset

Use case

Before you can measure anything, you need realistic test cases. Writing these by hand doesn't scale, so this notebook uses Claude to *generate its own test dataset*.

Problem Statement

Implement a AI Tool to generate AWS Specific Code generation for any given tasks

What it does

- Sends Claude a meta-prompt asking it to produce 3 AWS-flavored tasks, each solvable with a single Python function, JSON object, or regex.
- Parses the JSON response and saves it to `dataset.json`.

Example output

```
[
  { "task": "Create a regular expression to validate an AWS IAM username..." },
  { "task": "Write a Python function that converts an EC2 instance type into its
compute category..." },
  { "task": "Generate a JSON configuration for an AWS Lambda function..." }
]
```

Why it matters

This is Step 2 of the eval workflow in isolation — proving you can generate a scalable, varied test set without manually writing dozens of examples. In a real project this same technique generates hundreds of cases.

Notebook 2 — `002_run_the_eval.ipynb`: Wiring Up the Pipeline

Use case

You have test cases — now you need the plumbing that feeds each one through your prompt and collects the results, *before* worrying about how to score them.

What it does

- `run_prompt(test_case)` — merges each task into the prompt template and sends it to Claude.
- `run_test_case(test_case)` — calls `run_prompt`, then returns a result object with a **hardcoded score of 10** (a placeholder — grading isn't built yet).
- `run_eval(dataset)` — loops over every test case and collects results into a list.

Why it matters

This isolates "does my pipeline run end-to-end" from "is my grading any good." It's a deliberate checkpoint: get the mechanics — load dataset → call Claude → collect structured results — working first, so that when grading breaks later, you know it isn't the plumbing. The observed problem at this stage (verbose, unformatted Claude output) is exactly what later notebooks fix.

Notebook 3 — `003_model_grader.ipynb`: Grading with an LLM Judge

Use case

A hardcoded score of 10 tells you nothing. This notebook introduces the **model grader** — using a second Claude call as an automated judge of response quality.

What it does

- `grade_by_model(test_case, output)` — sends the task + Claude's solution to Claude again, asking it to return `strengths`, `weaknesses`, `reasoning`, and a `score` (1–10) as structured JSON.
- Updates `run_test_case` to call the grader and attach the score + reasoning to each result.
- `run_eval` now computes the **average score** across the dataset using `statistics.mean`.

Example result

```
{
  "output": "^[a-zA-Z0-9_-]{1,64}$",
  "test_case": { "task": "Create a regular expression to validate an AWS IAM
username..." },
  "score": 8,
  "reasoning": "The regex correctly captures most IAM username requirements..."
}
```

```
Average score: 7.33
```

Why it matters

This gives you your first *objective, numeric* signal — asking for reasoning alongside the score (rather than just a number) is what keeps the judge from defaulting to a lazy "6 out of 10" for everything, per 05_Model_based_grading.md.

Notebook 4 — `004_code_grader.ipynb`: Adding Deterministic Syntax Checks

Use case

A model grader is flexible but subjective — it might rate broken code highly if the reasoning "sounds" convincing. This notebook adds a **code grader**: a cheap, deterministic check that the output actually *parses* as valid Python, JSON, or regex.

What it does

- `validate_json`, `validate_python`, `validate_regex` — each tries to parse the raw output and returns `10` (valid) or `0` (invalid syntax error).
- `grade_syntax(response, test_case)` — dispatches to the right validator based on a new `"format"` field in each dataset entry.
- The prompt itself is tightened: explicit instructions ("respond only with Python, JSON, or a plain Regex, no commentary") plus assistant prefilling with ```` ```code ```` to force Claude to emit raw code instead of an explanation.
- Final score = `(model_score + syntax_score) / 2` — content quality and technical correctness are weighted equally.

Example result

```
{
  "output": "^i-[a-zA-Z0-9]+$",
  "test_case": { "task": "Design a regular expression to validate a valid AWS EC2
instance ID...", "format": "regex" },
  "score": 8.0,
  "reasoning": "The regex captures the basic structure of an EC2 instance ID but
lacks precision..."
}
```

`Average score: 8.17` — up from 7.33, showing the tighter prompt improved results.

Why it matters

This is the combined-grading approach from 06_Code_based_grading.md: code graders catch objective failures (invalid syntax) cheaply and reliably, while model graders assess subjective quality. Together they're more trustworthy than either alone — and the score jump between notebooks 3 and 4 is a concrete demonstration of "prompt change → measurable improvement."

Notebook 5 — `005_prompt_eval_excercise.ipynb`: Criteria-Aware Grading

Use case

Generic model grading ("is this a good solution?") can be inconsistent because the judge doesn't know what "good" means for a specific task. This exercise notebook makes grading **task-aware** by grounding the judge in explicit success criteria.

What it does

- The dataset generation prompt now also asks for a `"solution_criteria"` field per task — the specific standard the solution should be judged against.
- `grade_by_model` is updated to inject `test_case["solution_criteria"]` into the evaluation prompt inside a `<criteria>` block, so the judge scores against defined requirements rather than open-ended judgment.
- The rest of the pipeline (code grader, combined scoring, `run_eval`) is unchanged from notebook 4.

Example dataset entry

```
{
  "task": "Create a JSON configuration for an AWS Lambda function that sets
environment variables for database connection",
  "format": "json",
  "solution_criteria": "Key criteria for evaluating the solution"
}
```

Why it matters

This is the natural next refinement after notebook 4: once you trust the mechanics of grading, the highest-leverage improvement is making the *judge* more precise, not just the *generator*. It shows learners how to reduce grading noise/variance by anchoring the model grader to per-task rubrics instead of one generic prompt for every task type.
